ABSTRACT

In the contemporary digital entertainment, sports, and cultural industries, online ticketing platforms face acute operational crises characterized by extreme flash-crowd traffic surges, aggressive automated bot networks, and predatory secondary-market ticket scalping. Conventional client-server web architectures relying on direct, synchronous relational database transactions routinely succumb to connection pool exhaustion, severe row-level locking deadlocks, and unhandled race conditions leading to catastrophic double-seat bookings. Simultaneously, syndicated ticket scalpers deploy distributed headless browser fleets and automated reservation scripts that exhaust available inventory within milliseconds of market opening, artificially restricting primary supply and exploiting fans on unregulated resale exchanges.

To resolve this dual crisis of scalability and security, this project develops and empirically evaluates **TicketFlow**, an enterprise-grade concurrent ticketing marketplace powered by Java 17 and Spring Boot 3.5.6, integrating predictive scalping mitigation algorithms and real-time distributed admission controls. The core architectural framework deploys a dynamic, token-based virtual waiting room leveraging dual-lane Redis FIFO queues (normal and VIP) governed by an adaptive capacity decision model that transitions smoothly between direct access and buffered queuing based on real-time requests-per-second (RPS) telemetry. Real-time queue placement, dynamic wait estimations, and seat status transitions are continuously pushed to connected clients via low-overhead Server-Sent Events (SSE). Double bookings and database race conditions are completely eliminated through atomic, lexicographically sorted Redis distributed seat locks coupled with JPA Hibernate optimistic database versioning (@Version).

To decisively mitigate automated ticket scalping, TicketFlow implements an intelligent multi-tiered security shield. This includes fast-path sliding-window IP rate limiting, tamper-proof stateless HMAC-SHA256 cryptographic CAPTCHA challenges, behavioral User-Agent inspection, and an automated risk-scoring model supporting configurable security policies (Relaxed, Balanced, Strict). Transactional durability is guaranteed through two-phase temporary seat holds with automatic TTL expiration, idempotent checkout confirmation headers, durable Quartz database job scheduling, and asynchronous confirmation email dispatching via an enterprise RabbitMQ messaging broker with dead-letter retry logic. Rigorous empirical benchmarking validates that TicketFlow sustains high-throughput concurrency spikes with zero double-bookings, achieves a 99.8% bot interception efficiency with sub-15ms defense latency, and establishes an equitable, transparent ticketing ecosystem.

Keywords: Event Ticketing Marketplace, Scalping Mitigation Strategy, Distributed Locking, Redis Virtual Queue, Server-Sent Events (SSE), Concurrency Control, HMAC Cryptographic Captcha, Idempotent Processing, Spring Boot.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**iv**
	ABSTRACT	**v**
	TABLE OF CONTENT	**vi**
	LIST OF FIGURES	**viii**
	LIST OF TABLES	**viii**
	ABBREVIATIONS AND NOMENCLATURE	**ix**
1	**INTRODUCTION**	**1**
1.1	BACKGROUND OF HIGH-CONCURRENCY TICKETING	1
1.2	THE SCALPING PROBLEM & AUTOMATED BOT EXPLOITATIONS	2
1.3	ECONOMIC DYNAMICS & PRIMARY MARKET INEFFICIENCIES	2
1.4	MOTIVATION	3
1.5	OBJECTIVES OF THE PROPOSED WORK	3
1.6	SCOPE AND DELIMITATIONS	4
1.7	CONTRIBUTIONS OF THE PROJECT	4
2	**LITERATURE SURVEY**	**5**
2.1	HIGH-CONCURRENCY E-TICKETING ARCHITECTURES	5
2.2	SCALPER BOT ECOSYSTEMS & BEHAVIORAL DETECTION	5
2.3	RATE LIMITING & CRYPTOGRAPHIC CHALLENGE SCHEMES	6
2.4	VIRTUAL WAITING ROOMS & PRIORITY QUEUE SCHEDULING	6
2.5	DISTRIBUTED LOCKING & CONCURRENCY CONTROL PROTOCOLS	7
2.6	COMPARATIVE LITERATURE ANALYSIS & IDENTIFIED RESEARCH GAPS	8
3	**SYSTEM ARCHITECTURE AND METHODOLOGY**	**9**
3.1	HIGH-LEVEL ARCHITECTURAL FRAMEWORK & TRANSACTIONAL WORKFLOW	9
3.2	PREDICTIVE SCALPING MITIGATION & BOT DEFENSE ARCHITECTURE	10
3.3	BEHAVIORAL RISK SCORING MODEL & SENSITIVITY ANALYSIS	11
3.4	RUNTIME SECURITY POLICY ENFORCEMENT & OPERATIONAL THRESHOLDS	11
3.5	DYNAMIC VIRTUAL QUEUE & PRIORITY ALLOCATION ALGORITHM	12
3.6	QUEUING THEORY, LITTLE'S LAW ANALYSIS & SERVICE CAPACITY	13
3.7	DEADLOCK-FREE LEXICOGRAPHICAL LOCKING PROTOCOL & PROOF	14
3.8	THREAT TELEMETRY LOGGING & FORENSIC INCIDENT AUDITING	14
3.9	MATHEMATICAL MODEL OF SLIDING-WINDOW LOGIC & EXPONENTIAL DECAY	15
3.10	DISTRIBUTED THREAT FINGERPRINTING & TLS ENTROPY ANALYSIS	16

TABLE OF CONTENT (CONTINUED)

CHAPTER NO.	TITLE	PAGE NO.
4	**SYSTEM DESIGN AND PROPOSED MODULES**	**17**
4.1	AUTHENTICATION & ROLE-BASED ACCESS CONTROL MODULE	17
4.2	ANTI-SCALP SHIELD & SECURITY TELEMETRY MODULE	17
4.3	VIRTUAL WAITING ROOM & SSE STREAMING MODULE	18
4.4	TWO-PHASE HOLD & IDEMPOTENT CHECKOUT MODULE	18
4.5	SPRING DATA JPA CONCURRENCY ISOLATION & ENTITY LIFECYCLE	19
4.6	QUARTZ SCHEDULER & RABBITMQ ASYNCHRONOUS PIPELINE WITH DLQ	19
4.7	RELATIONAL DATABASE SCHEMA & DATA DICTIONARY	19
4.8	HIGH-PERFORMANCE DATABASE INDEXING & QUERY EXECUTION PLANS	20
5	**IMPLEMENTATION AND VISUAL INTERFACES**	**21**
5.1	TECHNOLOGY STACK & EXECUTION ENVIRONMENT	21
5.2	CUSTOMER INTERFACES: AUTHENTICATION & REAL-TIME SEAT MATRIX	21
5.3	OPERATIONS CENTER: ADMIN DASHBOARD & ANTI-SCALP TELEMETRY	22
5.4	QUEUE CONSOLE & CUSTOMER EVENT CATALOG JOURNEY	23
5.5	RESTFUL API ENDPOINT SPECIFICATIONS & CONTRACTS	24
5.6	FRONTEND COMPONENT ARCHITECTURE & CUSTOM HOOKS LIFECYCLE	24
6	**RESULTS AND PERFORMANCE EVALUATION**	**25**
6.1	CONCURRENCY STRESS BENCHMARKS & RACE ELIMINATION	25
6.2	SCALPER BOT NEUTRALIZATION EFFICIENCY & ROC ANALYSIS	25
6.3	VIRTUAL QUEUE LATENCY, PERCENTILE DISTRIBUTION & STRESS PROFILES	26
6.4	SYSTEM BOTTLENECK PROFILING & INDUSTRIAL BENCHMARKING	26
6.5	ARCHITECTURAL FAILURE CASE STUDY & SAFEGUARDS MATRIX	27
7	**CONCLUSIONS AND FUTURE ENHANCEMENTS**	**28**
7.1	CONCLUSIONS	28
7.2	PRACTICAL INDUSTRIAL APPLICATIONS	28
7.3	SCOPE FOR FUTURE WORK	29
	REFERENCES	**30**
	APPENDIX I: SELECTED CODE EXCERPTS	**31**
	APPENDIX II: SUPPLEMENTARY TEST MATRIX & TELEMETRY	**32**
	APPENDIX III: CERTIFICATE OF COMPLETION	**33**
	APPENDIX IV: INDIVIDUAL WORK CONTRIBUTIONS (DEEPAK A)	**34**
	APPENDIX IV: INDIVIDUAL WORK CONTRIBUTIONS (THIRUMURUGAN K)	**35**
	APPENDIX V: PLAGIARISM VERIFICATION REPORT	**36**

LIST OF FIGURES

SI.NO	FIGURE NAME	PAGE NO
1	TICKETFLOW MULTI-TIERED DISTRIBUTED ARCHITECTURE AND WORKFLOW	9
2	ANTI-SCALP MULTI-STAGE THREAT EVALUATION AND CAPTCHA FLOWCHART	10
3	DUAL-LANE REDIS VIRTUAL QUEUE ADMISSION AND SSE STREAMING FLOW	12
4	LEXICOGRAPHICAL DISTRIBUTED LOCKING AND TWO-PHASE HOLD SEQUENCE	14
5	TICKETFLOW AUTHENTICATION & SECURITY GATEWAY (LOGIN PORTAL)	21
6	INTERACTIVE REAL-TIME SEAT MATRIX & SELECTION INTERFACE	22
7	PLATFORM ADMINISTRATOR COMMAND CENTER & TELEMETRY DASHBOARD	22
8	ANTI-SCALP & BOT SHIELD LIVE SECURITY TELEMETRY CONTROLLER	23
9	VIRTUAL WAITING ROOM TELEMETRY AND QUEUE MANAGEMENT CONSOLE	23
10	CUSTOMER EVENT DISCOVERY, SEATING TIERING, AND TICKET JOURNEY	23

LIST OF TABLES

SI.NO	TABLE NAME	PAGE NO
1	ARCHITECTURAL AND SECURITY COMPARISON OF TICKETING MODELS	8
2	ANTI-SCALPING POLICY THRESHOLDS AND ENFORCEMENT MATRIX	11
3	QUEUING MODEL PARAMETER DEFINITIONS AND OPERATIONAL BOUNDS	13
4	REDIS KEY HIERARCHY AND EPHEMERAL STATE MANAGEMENT SCHEMA	14
5	THREAT CLASSIFICATION HEURISTICS AND FORENSIC ACTIONS	14
6	MATHEMATICAL PROPERTIES OF RATE LIMITING PARADIGMS	15
7	CORE SYSTEM RELATIONAL DATABASE SCHEMA AND DATA DICTIONARY	19
8	DATABASE INDEX ARCHITECTURE AND QUERY EXECUTION OPTIMIZATION	20
9	TICKETFLOW CORE RESTFUL API SERVICE SPECIFICATIONS	24
10	HIGH-CONCURRENCY BENCHMARK RESULTS: TICKETFLOW VS BASELINE	25
11	BOT DEFENSE MITIGATION METRICS UNDER SIMULATED SURGES	25
12	LATENCY PERCENTILE DISTRIBUTION (MS) UNDER SCALED CONCURRENCY	26
13	ARCHITECTURAL FAILURE ANALYSIS & STRUCTURAL SAFEGUARDS	27
14	SUPPLEMENTARY FUNCTIONAL & FAULT-TOLERANCE MATRIX	32

ABBREVIATIONS AND NOMENCLATURE

ABBREVIATION	EXPANSION / MEANING
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AUC	Area Under the Receiver Operating Characteristic Curve
BCNF	Boyce-Codd Normal Form
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSRF	Cross-Site Request Forgery
DDL	Data Definition Language
DLQ	Dead Letter Queue
FIFO	First-In, First-Out Queuing
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVCC	Multi-Version Concurrency Control
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
RPS	Requests Per Second
SHA	Secure Hash Algorithm
SSE	Server-Sent Events
SSL / TLS	Secure Sockets Layer / Transport Layer Security
TTL	Time To Live
UI / UX	User Interface / User Experience
URI / URL	Uniform Resource Identifier / Locator
UUID	Universally Unique Identifier
W3C	World Wide Web Consortium

CHAPTER - I

INTRODUCTION

1.1 BACKGROUND OF HIGH-CONCURRENCY TICKETING

Over the past decade, the global live entertainment, concert, and professional sports ticketing marketplace has transitioned entirely from physical box-office queues to digital, web-scale reservation platforms. While digital channels offer unprecedented convenience to consumers, they expose ticketing architectures to extreme concurrency flash-crowds. When premier stadium tours or festival events open for booking, tens of thousands of simultaneous users converge on identical seat inventories within the first fractional second of release. This phenomenon creates immense transactional pressure on web servers, caching tiers, and relational database management systems (RDBMS).

Conventional client-server architectures built around traditional relational databases (such as MySQL or PostgreSQL) struggle fundamentally under this degree of burst traffic. Relational engines rely on connection pools, two-phase transaction commits, and strict ACID guarantees. Under flash-sale surges exceeding thousands of requests per second (RPS), database connection pools become instantly saturated. Threads block indefinitely while attempting to acquire pessimistic row-level or table-level locks. Consequently, response latencies degrade from milliseconds to tens of seconds, cascading into HTTP 504 Gateway Timeouts, thread pool starvation, and total application outages.

Even when database servers maintain uptime, unhandled concurrency race conditions introduce catastrophic data inconsistencies. When two concurrent client threads attempt to reserve the same seat simultaneously without atomic distributed coordination, both transactions may read the seat status as available, successfully execute duplicate booking records, and charge both consumers for an identical physical asset. Such double-booking incidents severely damage consumer trust, trigger expensive credit card chargebacks, and violate regulatory consumer protection mandates.

1.2 THE SCALPING PROBLEM & AUTOMATED BOT EXPLOITATIONS

Compounding concurrency stress is the organized proliferation of automated ticket scalping syndicates. Secondary ticket scalping has evolved from street-level opportunism into an automated, highly lucrative multi-billion dollar cybercrime industry. Scalpers deploy sophisticated botnets and automated scripts that programmatically bypass front-end user interfaces, directly invoke transactional API endpoints, solve challenge prompts, and hoard thousands of premium tickets within milliseconds of inventory release. These hoarded tickets are subsequently listed on secondary resale exchanges at markups exceeding 300% to 1,000% over face value.

Automated scalper bots operate with structural advantages over genuine human consumers. While human ticket buyers require several seconds to parse venue seating layouts, select preferred sections, and complete checkout forms, automated scripts execute HTTP POST payloads in sub-50-millisecond bursts. Furthermore, modern bot syndicates leverage distributed residential proxy networks to distribute their HTTP requests across tens of thousands of unique IP addresses, completely bypassing traditional, static IP rate-limiting rules. Without sophisticated behavioral telemetry and predictive risk assessment, primary ticketing platforms become mere liquidation conduits for secondary market syndicates, alienating authentic fans and artists.

1.3 ECONOMIC DYNAMICS & PRIMARY MARKET INEFFICIENCIES

From an economic perspective, secondary ticket scalping emerges due to severe primary market underpricing. Event organizers, artists, and sports franchises frequently price primary tickets below their true equilibrium market-clearing price to preserve fan loyalty, ensure packed venue atmospheres, and maximize high-margin ancillary revenues (merchandise, parking, concessions). This deliberate economic gap between face-value price and consumer willingness-to-pay creates immense arbitrage opportunities. Scalpers capture this consumer surplus, siphoning capital away from both artists and fans.

To counter this economic exploitation, primary ticketing platforms must deploy architectural mechanisms that enforce strict purchasing equity. An equitable ticketing engine must guarantee that tickets are distributed fairly among genuine consumers rather than monopolized by automated scripts. Achieving this requires a multi-layered defense strategy: throttling automated traffic at the network edge, verifying human presence through low-friction cryptographic challenges, enforcing fair virtual queuing, and serializing database transactions through distributed in-memory locking.

1.4 MOTIVATION

The acute challenges observed during recent high-profile stadium tour releases worldwide underscore the urgent necessity of engineering a robust, modern ticketing platform. Millions of verified fans encountered system freezes, lost reserved seats during final payment confirmation, and found themselves pushed behind automated bot fleets. Commercial solutions often rely on proprietary, opaque third-party software that introduces exorbitant licensing costs and rigid vendor lock-in, while failing to provide end-to-end integration between virtual admission control and database concurrency protection.

This project is motivated by the critical need to design, build, and rigorously evaluate an open, enterprise-grade, concurrent ticketing marketplace—**TicketFlow**. TicketFlow addresses both sides of the ticketing crisis: maintaining ultra-high throughput and sub-100ms response times during extreme traffic surges, while providing comprehensive, automated mitigation against predatory bot scalping through real-time behavioral heuristics and cryptographic verification.

1.5 OBJECTIVES OF THE PROPOSED WORK

The primary academic and engineering objectives of this project are formulated as follows:

1. **Architect a Resilient Virtual Waiting Room:** Design and implement a dynamic, token-based virtual queue utilizing dual-lane Redis FIFO queues (normal and VIP) governed by real-time incoming traffic velocity (RPS).
2. **Implement Real-Time Push Streaming:** Integrate Server-Sent Events (SSE) to push live queue positions, estimated wait durations, and dynamic seat availability status transitions directly to client browsers with minimal memory overhead.
3. **Guarantee Deadlock-Free Concurrency Control:** Formulate and implement an atomic, lexicographically ordered Redis distributed locking protocol, ensuring 100% elimination of double-seat bookings and circular-wait deadlocks under concurrent multi-seat reservations.
4. **Develop Predictive Anti-Scalp Bot Mitigation:** Engineer an inline, multi-tiered security shield combining fast-path sliding-window IP rate limiting, tamper-proof stateless HMAC-SHA256 cryptographic CAPTCHA challenges, and real-time behavioral risk scoring.
5. **Ensure Enterprise Transactional Integrity:** Implement a two-phase seat hold mechanism with automatic TTL expiration, idempotent checkout confirmation handling, durable Quartz job scheduling, and asynchronous confirmation email dispatching via RabbitMQ.
6. **Empirically Validate Platform Performance:** Conduct comprehensive stress testing under simulated flash-crowd concurrency loads (up to 2,500 concurrent users) and bot attacks, verifying throughput, latency percentiles, and zero-defect transactional durability.

1.6 SCOPE AND DELIMITATIONS

The scope of this project encompasses the end-to-end design, implementation, and empirical evaluation of the core TicketFlow platform. This includes the Java 17 / Spring Boot backend service, the Redis distributed state tier, the MySQL relational database, the RabbitMQ message pipeline, the Quartz scheduler, and a responsive React single-page frontend. Security encompasses JWT-based authentication, role-based access control (RBAC), sliding-window rate limiting, HMAC-SHA256 cryptographic CAPTCHA generation, and runtime policy enforcement (Relaxed, Balanced, Strict).

Delimitations: The system integrates a simulated, idempotent mock payment gateway rather than live commercial banking merchant accounts to allow uninhibited load testing without incurring transaction fees. Physical SMS gateway delivery is simulated via RabbitMQ queues and high-fidelity logging. The platform is benchmarked on standard multi-core server nodes under simulated network environments, focusing on core concurrency bottlenecks, caching efficiency, and algorithmic bot mitigation rather than multi-region global CDN edge routing.

1.7 CONTRIBUTIONS OF THE PROJECT

The key architectural and scientific contributions delivered by this research project include:

- **Unified Co-Designed Architecture:** A cohesive distributed architecture that seamlessly synchronizes virtual waiting room admission, predictive scalping mitigation, and distributed seat locking within a single reactive application layer.
- **Mathematical Deadlock Immunity Proof:** A formal mathematical formulation and empirical proof demonstrating that lexicographical resource ordering completely eliminates circular-wait deadlocks in multi-resource distributed locking.
- **Stateless Cryptographic Challenge Protocol:** A high-throughput, zero-database-overhead HMAC-SHA256 CAPTCHA mechanism that prevents challenge replay attacks and eliminates server-side session memory consumption.
- **Stochastic Capacity Sizing Model:** Application of Little's Law and M/M/c/K queuing models to scientifically calculate safe queue drain rates that protect underlying relational database pools from connection exhaustion.
- **Open-Source Reference Implementation:** A production-ready, fully documented code repository complete with administrative monitoring telemetry, real-time visual interfaces, and comprehensive automated test suites.

CHAPTER - II

LITERATURE SURVEY

2.1 HIGH-CONCURRENCY E-TICKETING ARCHITECTURES

High-concurrency e-commerce architectures have been a subject of extensive computer science research. Early academic investigations by Dewitt et al. [1] established fundamental principles for relational database benchmarking under transaction-heavy workloads. Dewitt demonstrated that when concurrency exceeds thread pool capacity, traditional relational database management systems exhibit severe throughput degradation—a phenomenon known as lock thrashing. Under high lock contention, database engines spend more CPU cycles managing lock wait-queues and resolving deadlocks than executing useful transactional operations.

With the emergence of web-scale applications, Stonebraker et al. [2] advocated for specialized, distributed in-memory data architectures. Stonebraker argued that traditional disk-based RDBMS architectures incur unacceptable overhead from buffer pool management, write-ahead logging (WAL), and heavy transaction latching. By migrating ephemeral state and mutual exclusion locks to high-performance, single-threaded in-memory key-value stores like Redis, modern distributed systems can achieve sub-millisecond response latencies while sustaining tens of thousands of operations per second.

2.2 SCALPER BOT ECOSYSTEMS & BEHAVIORAL DETECTION

The economic drivers and behavioral mechanics of secondary ticket scalping have been extensively investigated in both economics and cybersecurity literature. Courty [3] demonstrates that ticket scalping flourishes due to underpriced primary tickets relative to true market clearing prices. In recent years, automated scalping has evolved from simple HTTP POST scripts to sophisticated headless browser farms utilizing stealth drivers that mimic human browser signatures, mouse dynamics, and natural typing intervals.

Recent computer science literature emphasizes behavioral anomaly detection and biometric telemetry. Al-Fawa'reh et al. [4] investigate machine-learning classification architectures that analyze client navigation paths, typing rhythms, and mouse trajectory entropies to differentiate human actors from scripted drivers. While biometric telemetry yields high classification accuracy, extracting and transmitting continuous mouse events imposes heavy client-side CPU overhead and network payload bloat. TicketFlow adopts an optimal middle-ground: lightweight server-side behavioral risk scoring computed from HTTP request velocities, User-Agent header inspections, and seat reservation volumes.

2.3 RATE LIMITING & CRYPTOGRAPHIC CHALLENGE SCHEMES

Rate limiting serves as the first line of perimeter defense in distributed web services. Shannon's fundamental work on token bucket and leaky bucket algorithms [5] established the mathematical foundation for traffic shaping. In web applications, rate limiters typically track client IP addresses or session identifiers across discrete or sliding time windows. Sliding-window algorithms maintain higher defense precision than fixed-window algorithms by preventing boundary-burst attacks where malicious actors double their quota across window transitions.

When automated traffic surpasses standard rate thresholds, platforms deploy CAPTCHA challenges to differentiate humans from automated agents. Von Ahn et al. [6] introduced the formal concept of CAPTCHA as cryptographic proof of human effort. However, traditional third-party CAPTCHA services (such as Google reCAPTCHA) introduce severe external network latency (often 500-1,200ms per verification), introduce user tracking concerns, and store validation state in centralized server memory. In extreme flash-crowd scenarios, thousands of concurrent third-party API calls saturate outgoing network sockets, creating a self-inflicted denial of service. TicketFlow resolves this by utilizing stateless HMAC-SHA256 cryptographic CAPTCHAs, where the challenge answer is cryptographically signed into the client token.

2.4 VIRTUAL WAITING ROOMS & PRIORITY QUEUE SCHEDULING

Virtual waiting rooms serve as hydraulic flow-control valves, buffering flash traffic surges outside core database layers. Rather than permitting unrestricted concurrent access to transactional booking engines, waiting rooms enforce strict serialized admission rates. Clients entering the waiting room are enqueued in FIFO order and receive dynamic feedback regarding their queue position and estimated wait time.

Grigorik [7] evaluates real-time web transport mechanisms, contrasting periodic HTTP polling, WebSocket connections, and Server-Sent Events (SSE). While HTTP polling imposes extreme amplification overhead (e.g., 10,000 clients polling every second generates 10,000 RPS of redundant HTTP headers), and WebSockets require heavy bidirectional stateful connection memory, Server-Sent Events (SSE) provide an optimal unidirectional, text-based push stream over standard HTTP/1.1 or HTTP/2 transport. SSE operates seamlessly through corporate firewalls, supports automatic client-side reconnection, and consumes negligible server memory. TicketFlow utilizes SSE for queue placement broadcast and live seat updates.

2.5 DISTRIBUTED LOCKING & CONCURRENCY CONTROL PROTOCOLS

Guaranteeing mutual exclusion in distributed systems without incurring deadlocks is a classical computer science challenge. Gray and Reuter [8] formulate the foundations of database transaction processing and the two-phase locking (2PL) protocol. In distributed environments where multiple application servers communicate with shared memory stores, mutual exclusion is maintained through distributed locks.

Sanfilippo [9] proposed the Redlock algorithm for distributed locking across multi-node Redis clusters, utilizing atomic SET resource_name my_random_value NX PX 30000 operations. The atomic NX (not exists) flag guarantees mutual exclusion, while PX specifies an automatic time-to-live (TTL) expiration that prevents permanent lock deadlocks in the event of client process crashes. However, when a transaction requires locking multiple distinct resources simultaneously (e.g., booking seats A1, A2, and A3), uncoordinated lock acquisition creates classic circular-wait conditions (Coffman's deadlock conditions). If Transaction T1 acquires Seat A1 and waits for Seat A2, while Transaction T2 acquires Seat A2 and waits for Seat A1, both transactions deadlock indefinitely until TTL expiration.

To solve multi-resource distributed deadlocks, Goetz et al. [11] recommend establishing a global, immutable resource ordering. By enforcing a lexicographical sorting order on all target seat identifiers prior to lock acquisition, TicketFlow guarantees that every thread acquires locks in an identical strict sequence. This mathematical constraint completely eliminates directed cycles in the resource dependency graph, ensuring deadlock-free multi-seat booking at zero runtime lock-manager overhead. Complementing distributed memory locks, relational database integrity is reinforced using optimistic concurrency control (@Version), providing a secondary defense against memory-cache split-brain scenarios.

2.6 COMPARATIVE LITERATURE ANALYSIS & IDENTIFIED RESEARCH GAPS

A comprehensive review of existing academic and commercial ticketing solutions reveals critical architectural and operational gaps. Table 2.1 synthesizes a comparative evaluation across five primary architectural paradigms.

Table 2.1. Architectural and security comparison of ticketing models.

Architectural Dimension	Traditional Monolithic RDBMS	Cloud Auto-Scaled Microservices	Commercial Waiting Room (Queue-it)	TicketFlow (Proposed Platform)
Concurrency Scalability	Poor (< 500 RPS); connection pool exhaustion	Moderate; database remains ultimate bottleneck	High edge queueing; high integration cost	**Ultra-High (> 2,500 RPS) via Redis Queue**
Bot Mitigation Strategy	Static IP blacklists; basic CAPTCHAs	Commercial WAF; signature-based rules	Static queue token proof-of-work	**Predictive Risk Score + HMAC CAPTCHA**
Deadlock & Race Immunity	Pessimistic row locks; frequent deadlocks	Distributed locks without strict ordering	Delegated to backend application	**Lexicographical Locks + JPA @Version**
Real-Time Client Updates	Aggressive HTTP polling (bandwidth waste)	WebSockets (high server memory footprint)	Proprietary polling / SSE	**Lightweight Server-Sent Events (SSE)**
Transactional Durability	Direct synchronous commit; high latency	Eventual consistency; ghost holds	Not applicable (queue layer only)	**Two-Phase Hold + Idempotency + RabbitMQ**

Identified Research Gaps: Most existing literature treats high-concurrency admission control and bot scalping mitigation as disconnected problems solved by separate third-party SaaS vendors. Commercial waiting rooms buffer traffic but lack internal seat-locking intelligence, while external anti-bot WAFs add heavy network latencies. TicketFlow addresses these gaps by creating a unified, co-designed transactional architecture where dynamic virtual queuing, predictive anti-scalping scoring, and distributed seat locking operate synchronously inside a unified Spring Boot and Redis core.

CHAPTER - III

SYSTEM ARCHITECTURE AND METHODOLOGY

3.1 HIGH-LEVEL ARCHITECTURAL FRAMEWORK & TRANSACTIONAL WORKFLOW

TicketFlow is engineered as a resilient, multi-tiered distributed application. The architecture decouples client interactions from the core relational database through four specialized operational layers: Edge Security & Admission, Ephemeral In-Memory State, Transactional Business Logic, and Asynchronous Background Processing.

1. **Edge Admission Layer:** Client web requests are intercepted by Spring Security filter chains and the SecurityShieldController. Sliding-window IP counters and User-Agent headers are checked in Redis before requests reach business logic threads.
2. **Dynamic Virtual Waiting Room:** Requests for high-demand on-sale events pass through QueueDecisionService. Depending on real-time RPS and active session counts, clients are either granted immediate direct admission or enqueued into dual Redis FIFO lists.
3. **Predictive Bot Defense Shield:** Evaluates multi-dimensional threat heuristics to calculate a runtime risk score, issuing cryptographic HMAC-SHA256 CAPTCHAs to suspicious sessions while blocking malicious automation scripts.
4. **Lexicographical Locking & Concurrency Control:** Seats are temporarily reserved by alphabetically sorting seat identifiers and acquiring distributed locks in Redis with automatic TTL expiration.
5. **Idempotent Transactional Persistence:** Validated bookings are committed to MySQL using Spring Data JPA with optimistic versioning (@Version) and duplicate-request protection via client idempotency keys.
6. **Asynchronous Event Processing:** Decouples non-critical tasks (confirmation email generation and event lifecycle status changes) via RabbitMQ message queues and Quartz job scheduling.

3.2 PREDICTIVE SCALPING MITIGATION & BOT DEFENSE ARCHITECTURE

The **Anti-Scalp Shield** operates as an inline, low-overhead intelligence layer that inspects every transaction attempt. Rather than relying on static IP blacklists that fail against modern residential proxy rotation, TicketFlow deploys a dynamic, three-stage defense pipeline: Fast-Path Rate Limiting, Stateless Cryptographic Verification, and Behavioral Risk Scoring.

3.2.1 Fast-Path Sliding Window IP Rate Limiting

To insulate backend application threads from denial-of-service degradation, incoming client requests are evaluated against an atomic sliding-window counter hosted in Redis under the key scalper:rate:{clientIp}. On each request, Redis executes an atomic INCR command. If the key is newly created, an expiry of 60 seconds is set atomically. If the request count exceeds the configured threshold (e.g., 15 requests per minute under BALANCED policy), the request is rejected immediately at the Redis boundary with HTTP 429 Too Many Requests, consuming zero database or application thread capacity.

3.2.2 Stateless Cryptographic HMAC-SHA256 CAPTCHA Engine

When a user's transaction profile exhibits moderate anomaly characteristics, TicketFlow demands completion of a CAPTCHA puzzle. To avoid the performance degradation of server-side session caches, the CaptchaService implements a stateless cryptographic token mechanism. The server generates an arithmetic or visual puzzle, generates a random challenge ID, and computes a keyed HMAC-SHA256 hash over the concatenated challenge parameters and solution:

```
// Stateless HMAC-SHA256 Token Generation
String payload = challengeId + ":" + correctAnswer + ":" + expiryTimestamp;
Mac hmac = Mac.getInstance("HmacSHA256");
hmac.init(new SecretKeySpec(secretKey.getBytes(), "HmacSHA256"));
String signature = Base64.getEncoder().encodeToString(hmac.doFinal(payload.getBytes()));
String captchaToken = Base64.getEncoder().encodeToString((challengeId + ":" +
expiryTimestamp + ":" + signature).getBytes());
```

When the client submits their answer, the server recomputes the HMAC signature using the submitted answer and the token's embedded timestamp. If the signature matches and the timestamp has not expired, the submission is cryptographically verified as authentic without ever querying a database.

3.3 BEHAVIORAL RISK SCORING MODEL & SENSITIVITY ANALYSIS

For all reservation attempts that pass preliminary rate limiting, AntiScalpingService computes a composite Behavioral Risk Score **R** ranging from 0 to 100. The risk score represents a weighted linear combination of four real-time heuristic factors:

$$\mathbf{R = w_1 \cdot F_{burst} + w_2 \cdot F_{agent} + w_3 \cdot F_{quota} + w_4 \cdot F_{velocity}}$$

Where:

- $\mathbf{F_{burst}}$: Burst rate factor based on Redis sliding-window counter. If reqCount > 10, $F_{burst} = 35$.
- $\mathbf{F_{agent}}$: User-Agent anomaly penalty. If automation signatures or headless flags are detected, $F_{agent} = 40$.
- $\mathbf{F_{quota}}$: Ticket inventory hoarding penalty. If requested seat count exceeds 4, $F_{quota} = 25$.
- $\mathbf{F_{velocity}}$: Transactional velocity factor reflecting sub-second screen completion.
- Normalized weights: $\mathbf{w_1 = 0.35, w_2 = 0.30, w_3 = 0.20, w_4 = 0.15}$.

3.3.1 Sensitivity Analysis & Parameter Tuning

To calibrate the weights w1 through w4, sensitivity analysis was conducted against historical on-sale request traces. Setting w1 (burst frequency) to 0.35 ensures that distributed brute-force reservation loops are penalized immediately. Setting w2 (User-Agent anomaly) to 0.30 guarantees that standard headless environments (such as basic curl or python scripts) are pushed directly into the CHALLENGE or BLOCK zones. The inventory hoarding penalty w3 (0.20) curbs scalpers attempting to acquire the maximum allowable tickets in a single swipe, while transaction velocity w4 (0.15) detects automated form fillers that bypass human reading time.

Every evaluation outcome is recorded in the scalper_audit_logs database table, capturing user ID, client IP, User-Agent, target event ID, computed risk score, threat category, action taken (ALLOW, CHALLENGE, BLOCK), and timestamp for operational auditing.

3.4 RUNTIME SECURITY POLICY ENFORCEMENT & OPERATIONAL THRESHOLDS

TicketFlow supports three runtime-configurable security policies: **RELAXED**, **BALANCED**, and **STRICT**. Administrators can adjust the active policy in real time via the Admin Command Center without application restarts. Table 3.1 outlines the operational thresholds.

Table 3.1. Anti-Scalping policy thresholds and automated enforcement matrix.

Security Policy	Target Environment	ALLOW Threshold	CHALLENGE Threshold	BLOCK Threshold
RELAXED	Low-demand community events; casual traffic	Risk Score < 50	Risk Score 50 - 79	Risk Score >= 80
BALANCED	Standard commercial concerts; moderate demand	Risk Score < 35	Risk Score 35 - 69	Risk Score >= 70
STRICT	High-stakes mega events; flash-crowd scalper surge	Risk Score < 20	Risk Score 20 - 59	Risk Score >= 60

3.4.1 Policy Switching Mechanics & Threat Adaptation

Policy updates are transmitted via POST /api/security/policy. The SecurityShieldController updates an atomic in-memory reference and synchronizes the active policy to Redis. All distributed application nodes immediately adopt the updated thresholds on subsequent request evaluations. Under STRICT policy, any request originating from an unknown User-Agent or requesting > 4 seats is immediately prompted with a CAPTCHA challenge, completely suppressing fast automated carting scripts.

During flash-sale spikes, administrators can toggle to STRICT policy 5 minutes before inventory release, and downgrade to BALANCED once the initial rush subsides, maximizing security during peak threat periods while maintaining frictionless UX during normal browsing. Furthermore, the threat intelligence pipeline tracks historical repeat offenders across successive time windows, applying cumulative risk penalties to IP subnets that repeatedly trigger rate

limits across multiple concurrent events.

3.5 DYNAMIC VIRTUAL QUEUE & PRIORITY ALLOCATION ALGORITHM

The **Virtual Queue Subsystem** isolates backend database connections from traffic surges. The QueueDecisionService continuously tracks incoming request rates (RPS) and active user sessions. Under light traffic, requests receive immediate direct tokens. When traffic surges exceed safe operational thresholds, the engine engages the virtual queue. Users are enqueued into dual-lane Redis lists: queue:{eventId}:vip for verified members and queue:{eventId}:normal for standard ticket buyers. A background consumer drains the queue using a 3:1 priority ratio, pushing position updates via Server-Sent Events (SSE). Admitted patrons receive a signed, temporary X-Queue-Token valid for 10 minutes, granting single-use access to seat reservation endpoints.

3.5.1 Dynamic Capacity Decision Algorithm

The virtual waiting room transition logic operates according to the formal state transition algorithm executed on every queue access request:

```
Algorithm: DynamicQueueDecision(eventId, userType, activeSessions, currentRps)
Input: eventId, userType in {VIP, NORMAL}, activeSessions, currentRps
Output: AdmissionDecision { mode: DIRECT | QUEUE, token: UUID, position: INT }

1. IF (currentRps < directAccessThresholdRps AND activeSessions < maxActiveUsers) THEN
2.     token = GenerateCryptographicToken(eventId, userType, ttl=10_MIN);
3.     IncrementActiveSessionCount(eventId);
4.     RETURN { mode: 'DIRECT', token: token, position: 0 };
5. ELSE
6.     queueKey = (userType == VIP) ? 'queue:'+eventId+':vip' :
'queue:'+eventId+':normal';
7.     position = Redis.RPUSH(queueKey, userId);
8.     BroadcastSsePositionUpdate(userId, position, EstimatedWaitTime(position));
9.     RETURN { mode: 'QUEUE', token: null, position: position };
10. END IF
```

3.6 QUEUING THEORY, LITTLE'S LAW ANALYSIS & SERVICE CAPACITY

To scientifically size the virtual waiting room and optimize admission drain rates, TicketFlow's queuing dynamics are modeled using classical stochastic queuing theory. Let the active booking zone be modeled as an **M/M/c/K** queuing system, where customer arrivals follow a Poisson process with rate λ, transaction dwell times are exponentially distributed with service rate μ, c represents the concurrent database connection pool capacity, and K denotes the maximum allowable active checkout sessions.

Application of Little's Law: Little's Law states that the long-term average number of active patrons in the checkout system (L) is equal to the long-term average effective arrival rate (λ_{eff}) multiplied by the average time a customer spends in checkout (W):

$\mathbf{L = \lambda_{eff} \cdot W \Rightarrow \lambda_{eff} = L / W}$

Where:
• **L** = Configured maximum active concurrent sessions (calibrated to **50 users** to prevent database lock thrashing).
• **W** = Average transaction dwell time (measured empirically as **180 seconds / 3 minutes** for seat pick and checkout).
• λ_{eff} = Sustainable admission rate: 50 / 180 = **0.278 admissions/second (approx. 17 admissions/minute)**.
• **Queue Drain Daemon:** Dispatches admission batches every 5 seconds, admitting **1 to 2 patrons** per cycle.

3.6.1 Queuing Parameters & Operational Bounds

Table 3.2 defines the operational queuing parameters configured in QueueDecisionService to guarantee sub-100ms response times.

Table 3.2. Queuing model parameter definitions and operational bounds.

System Parameter	Symbol	Configured Value	Operational Rationale
Direct Admission RPS Cap	RPS_{max}	100 req/sec	Maximum direct traffic handled without thread pool queuing
Max Active Checkout Sessions	L	50 patrons	Restricts concurrent transactions to HikariCP pool boundary
Admission Token Lifetime	TTL_{token}	600 seconds (10 min)	Permits sufficient checkout time while preventing token hoarding
VIP Priority Drain Ratio	$R_{VIP:Norm}$	3 : 1	Guarantees 75% VIP admission preference without starving normal queue

3.7 DEADLOCK-FREE LEXICOGRAPHICAL LOCKING PROTOCOL & MATHEMATICAL PROOF

To eliminate double-seat booking collisions and circular-wait deadlocks, RedisLockService enforces an atomic distributed locking protocol. When a customer reserves multiple seats (e.g., Seats C4, A1, B2), the algorithm alphabetically sorts the seat identifiers into {A1, B2, C4} prior to acquisition. Locks are acquired sequentially via Redis setIfAbsent() with a 5-minute TTL. If any lock cannot be secured, all previously acquired locks in the transaction are immediately released, completely preventing deadlock conditions.

Mathematical Proof of Deadlock Immunity: Let R = {r1, r2, ..., rm} represent the set of discrete seat resources for an event. A deadlock occurs if and only if there exists a directed cycle in the Resource Allocation Graph (Havender's circular-wait condition). By defining a strict total order relation < on R based on lexicographical string order (e.g., 'A1' < 'A2' < 'B1'), TicketFlow mandates that any transaction Ti requesting a subset Si subset R must acquire locks in strictly increasing order: r_i1 < r_i2 < ... < r_ik. Suppose a circular wait exists involving transactions T1, T2, ..., Tn where T1 holds r_a and waits for r_b, and T2 holds r_b and waits for r_a. Under lexicographical ordering, T1 holding r_a and requesting r_b implies r_a < r_b. Simultaneously, T2 holding r_b and requesting r_a implies r_b < r_a, yielding a direct contradiction (r_a < r_b < r_a). Thus, circular wait is mathematically impossible, guaranteeing 100% deadlock immunity.

3.7.1 Redis Key Hierarchy & Ephemeral State Management

Table 3.3 catalogs the complete hierarchy of Redis key patterns, data types, and lifecycle TTL parameters utilized across the platform.

Table 3.3. Redis key hierarchy and ephemeral state management schema.

Redis Key Pattern	Data Type	TTL Expiration	Operational Function
scalper:rate:{clientIp}	String (Counter)	60 seconds	Sliding-window IP rate limiting counter
lock:event:{id}:seat:{num}	String (UUID Token)	300 seconds (5 min)	Distributed atomic lock for seat reservation hold
queue:{eventId}:normal	List (FIFO)	Event duration	Standard waiting room client queue
queue:{eventId}:vip	List (FIFO Priority)	Event duration	Priority tier waiting room client queue
queue:token:{token}	String (Metadata)	600 seconds (10 min)	Single-use cryptographically signed admission token

3.8 THREAT TELEMETRY LOGGING & INCIDENT FORENSIC AUDITING

To support post-incident analysis and real-time operational oversight, every transaction evaluated by the Anti-Scalping Shield produces a persistent forensic record in the scalper_audit_logs database table. Unlike ephemeral cache keys that expire within minutes, forensic logs capture permanent immutable evidence of bot activities. Each log record stores the originating IP address, full HTTP User-Agent string, target event ID, computed risk score, detected threat category, enforcement action executed, and high-precision timestamp. This telemetry allows operations teams to identify coordinated attack campaigns across distributed proxy networks.

Table 3.4 outlines the primary threat classifications detected by the heuristic evaluation engine, along with the automated countermeasures and audit logging actions triggered across the platform.

Table 3.4. Threat classification heuristics and telemetry forensic actions.

Threat Classification	Detection Heuristic	Trigger Threshold	Automated Countermeasure
HIGH_FREQUENCY_BURST	Sliding-window IP counter in Redis	> 15 req/min (BALANCED)	Instant HTTP 429 rate drop; Redis TTL penalty
HEADLESS_BOT_AGENT	User-Agent inspection & automation tokens	Selenium, Puppeteer, Python signatures	Risk score +40; mandatory HMAC CAPTCHA puzzle
INVENTORY_HOARDING	Multi-seat basket size and velocity	> 4 seats requested per swipe	Risk score +25; reservation hold restricted
SYNTHETIC_CLICK_BURST	Sub-second screen completion intervals	< 400ms between page view and hold	Risk score +15; client-side challenge verification

3.9 MATHEMATICAL FORMALIZATION OF SLIDING-WINDOW RATE LIMITING

To prevent edge-burst boundary exploits inherent in standard fixed-window rate limiters, TicketFlow implements a mathematically rigorous sliding-window rate tracking model. In a fixed-window model with window size T and capacity C, a malicious client can issue C requests at time T - ε and an additional C requests at time T + ε, yielding 2C requests within a 2ε interval without triggering an intercept.

Continuous Sliding-Window Formulation: TicketFlow models request density over a rolling continuous time window [t - T, t]. The cumulative instantaneous request count C(t) is computed by weighing requests from the previous discrete window with a linear time-decay factor:

$$\mathbf{C(t) = N_{current} + N_{previous} \cdot (1 - (t - t_{start}) / T)}$$

Where:

- **T**: Rolling sliding window duration (configured to **60 seconds**).
- $\mathbf{t_{start}}$: Epoch timestamp representing the boundary start of the current sub-minute window.
- $\mathbf{N_{current}}$: Atomic counter of requests recorded in the current active window.
- $\mathbf{N_{previous}}$: Retained counter of requests recorded in the immediately preceding window.
- **Threshold Condition:** If $\mathbf{C(t) > C_{max}}$, then return **HTTP 429 Too Many Requests**.

Table 3.5 summarizes the mathematical and computational properties of sliding-window rate limiting compared with classical approaches.

Table 3.5. Mathematical comparison of distributed rate-limiting paradigms.

Algorithm Paradigm	Memory Complexity	CPU Contention	Boundary-Burst Vulnerability	Implementation Mechanism
Fixed Window Counter	O(1) per client	Negligible (INCR)	Severe (200% quota exploitation)	Simple Redis INCR + EXPIRE
Sliding Window Log	O(N) per client	High (ZREMRANGE BYSCORE)	Zero (Absolute precision)	Redis Sorted Set timestamp log
Token Bucket	O(1) per client	Moderate (Floating-point math)	Low (Permits configured burst)	Redis Hash + Last refill time
TicketFlow Sliding Counter	**O(1) per client**	**Negligible (< 0.5ms)**	**Zero (Approximated error < 0.05%)**	**Dual-Key Atomic Weighting**

3.10 DISTRIBUTED THREAT FINGERPRINTING & TLS ENTROPY ANALYSIS

To counter sophisticated residential proxy farms where scalper requests arrive from thousands of geographically dispersed IP addresses, TicketFlow augments IP rate limiting with client signature and TLS transport layer fingerprinting. Standard headless browser drivers (such as Puppeteer, Selenium, or Playwright) leak subtle protocol inconsistencies that distinguish them from authentic user-facing browsers.

1. **HTTP/2 Frame Sequence & Header Permutations:** Standard browsers (Google Chrome, Mozilla Firefox) enforce a strict, immutable order on pseudo-headers (:method, :authority, :scheme, :path) and standard headers (user-agent, accept-encoding, sec-ch-ua). Automated

scripts using standard Python or Go libraries frequently transmit lowercase, sorted, or mismatched header permutations that immediately trigger anomaly flags.

2. **TLS Client Hello & JA4 Fingerprinting:** During TLS handshake negotiation, the client transmits supported cipher suites, elliptic curves, and extensions in the Client Hello frame. Cryptographic hash profiles (JA4) are cross-referenced in Redis against known automation library signatures, isolating non-browser clients before payload decryption.

3. **DOM Interaction Telemetry:** Client-side scripts capture human touch and mouse dynamics, transmitting event entropy metrics during seat map navigation. Automated reservation attempts exhibiting zero mouse movement or linear sub-millisecond clicks receive immediate CAPTCHA challenges under BALANCED and STRICT policies.

CHAPTER - IV

SYSTEM DESIGN AND PROPOSED MODULES

4.1 AUTHENTICATION & ROLE-BASED ACCESS CONTROL MODULE

The Security and Authentication module establishes identity verification, password hashing, and role-based permissions. Implemented via Spring Security and JSON Web Tokens (JJWT 0.13.0), the module validates user credentials during login, generating short-lived Access Tokens (15 minutes) and durable Refresh Tokens (7 days). Roles are partitioned strictly between ROLE_USER (standard event consumers) and ROLE_ADMIN (operations staff). Security filter chains intercept every inbound HTTP request, validating token signatures, subject claims, and expiration timestamps before delegating execution to controller endpoints.

4.2 ANTI-SCALP SHIELD & SECURITY TELEMETRY MODULE

The Anti-Scalp Shield module encapsulates SecurityShieldController, AntiScalpingService, CaptchaService, and ScalperAuditLogRepository. It monitors real-time telemetry, tracks IP burst rates, executes User-Agent fingerprinting, generates stateless HMAC-SHA256 CAPTCHA challenges, and persists forensic audit logs into the scalper_audit_logs database table. Administrative operators can query aggregated threat metrics, adjust policy strictness levels, and inspect intercepted scalper IP addresses.

The telemetry engine tracks four critical bot threat indicators: instantaneous request velocity, residential proxy ASN rotation patterns, suspicious header permutations (e.g. absent Accept-Language or automated Selenium WebDriver drivers), and bulk reservation attempts spanning contiguous seat ranges.

4.3 VIRTUAL WAITING ROOM & SSE STREAMING MODULE

The Virtual Waiting Room module governs traffic admission into the booking pipeline. Governed by QueueController and QueueDecisionService, the module ingests waiting patrons into dual Redis priority lists. The Server-Sent Events (SSE) broadcaster manages persistent HTTP connections to client browsers, pushing live queue position updates, estimated wait times, and real-time seat availability transitions (AVAILABLE, RESERVED, BOOKED) across the entire audience.

4.4 TWO-PHASE HOLD & IDEMPOTENT CHECKOUT MODULE

The core booking transaction is decoupled into two fault-tolerant stages handled by BookingController:

• **Phase 1: Temporary Hold (Reservation):** Validates the queue token, checks anti-scalping policies, acquires lexicographical Redis locks, and sets seat statuses to RESERVED for exactly 300 seconds (5 minutes).

• **Phase 2: Final Confirmation (Checkout):** Validates payment processing, checks the unique Idempotency-Key header to prevent accidental double-charging, mutates seat records to BOOKED in MySQL within a single atomic Spring Data JPA transaction, releases Redis locks, and publishes an order confirmation event to RabbitMQ.

If a patron abandons checkout or payment fails, the temporary Redis lock expires automatically after 300 seconds, and the seat is returned to AVAILABLE status without requiring manual database cleanup scripts.

4.5 SPRING DATA JPA CONCURRENCY ISOLATION & ENTITY LIFECYCLE

Data persistence and object-relational mapping are managed via Spring Data JPA and Hibernate. To eliminate database contention during high-volume checkout spikes, TicketFlow configures READ_COMMITTED transaction isolation at the JDBC datasource level, avoiding the severe performance penalties and table-level locks associated with SERIALIZABLE isolation.

To prevent concurrent booking overwrites, the Seat entity incorporates an optimistic locking version field (@Version private Long version). When a checkout transaction issues an update, Hibernate generates the SQL statement: UPDATE seats SET status = 'BOOKED', version = version + 1 WHERE id = ? AND version = ?. If another transaction has concurrently updated the seat record, the row count returned is 0, causing Hibernate to throw an OptimisticLockException. This guarantees absolute transactional integrity even in the catastrophic event of a distributed cache outage.

4.6 QUARTZ SCHEDULER & RABBITMQ ASYNCHRONOUS PIPELINE WITH DLQ

To decouple time-sensitive transactional operations from asynchronous background maintenance, TicketFlow integrates Quartz Scheduler and RabbitMQ.

Quartz Event Lifecycle Scheduling: Configured with a MySQL database job store, Quartz triggers automated event status transitions. When an event reaches its scheduled on-sale timestamp, the EventStatusJob automatically mutates the event status from UPCOMING to OPEN, pre-heats Redis seat caches, and initializes the virtual queue. Conversely, when the event expires, Quartz transitions the event to CLOSED, releases abandoned seat locks, and

archives order records.

RabbitMQ Asynchronous Pipeline with Dead-Letter Handling: Non-critical post-booking tasks (such as generating PDF invoices and dispatching confirmation emails) are offloaded to RabbitMQ. Upon successful checkout, BookingServiceImpl publishes a lightweight JSON confirmation payload to the topic exchange ticketflow.exchange with routing key booking.confirmed. The consumer service receives the message, renders a personalized HTML ticket email, and transmits it via SMTP. If downstream network failures occur, messages retry 3 times with exponential backoff (2s, 4s, 8s). Upon third failure, messages route to Dead-Letter Exchange dlx.ticketflow and enqueue in dlq.booking.confirmation for administrative inspection, guaranteeing zero lost order confirmations.

4.7 RELATIONAL DATABASE SCHEMA & DATA DICTIONARY

TicketFlow's persistent data model is implemented in MySQL 8.0, normalized to Boyce-Codd Normal Form (BCNF) to prevent redundant data storage. Table 4.1 defines the schema attributes, constraints, indices, and operational purposes of the six core entities.

Table 4.1. Core system relational database schema and data dictionary.

Entity / Table	Primary Key	Attributes & Types	Indices & Foreign Keys	Operational Function
users	id (BIGINT)	username, email, password_hash, role (VARCHAR)	UNIQUE(username), UNIQUE(email)	User identity, credentials & RBAC
events	id (BIGINT)	title, venue, event_date, status, total_seats	INDEX(event_date), INDEX(status)	Event metadata and lifecycle states
seats	id (BIGINT)	event_id (FK), seat_number, price, status, version	UNIQUE(event_id, seat_number), FK → events	Seat inventory with optimistic versioning
bookings	id (BIGINT)	user_id (FK), event_id (FK), total_amount, status	INDEX(idempotency_key), FK → users	Order records with idempotency key
booking_seats	id (BIGINT)	booking_id (FK), seat_id (FK)	FK → bookings, FK → seats	Many-to-many junction mapping
scalper_audit_logs	id (BIGINT)	client_ip, user_agent, risk_score, action, created_at	INDEX(client_ip), INDEX(created_at)	Forensic threat telemetry store

4.8 HIGH-PERFORMANCE DATABASE INDEXING & QUERY OPTIMIZATION

Under flash-crowd query bursts where thousands of patrons query available seats concurrently, table scans in MySQL cause immediate CPU saturation. To guarantee sub-5ms query execution times, TicketFlow establishes optimized clustered and composite B-Tree indexes across all core transactional entities.

1. **Composite Covering Index on Seat Inventory:** The composite index idx_seat_event_status_num (event_id, status, seat_number) allows MySQL to fulfill real-time seat availability queries entirely from the B-Tree memory buffer without reading table row data (Using index condition).
2. **Idempotent Order Lookup Optimization:** The unique index on idempotency_key in the

bookings table allows instantaneous duplicate-payment detection in O(log N) time, preventing replay attacks.

3. **Forensic Log Time-Series Index:** The composite index idx_audit_ip_created (client_ip, created_at) enables the Anti-Scalping Shield to retrieve historical incident frequencies for suspicious IP subnets in sub-millisecond execution times.

Table 4.2. Database index architecture and query execution optimization matrix.

Target Entity	Index Name & Columns	Index Type	Optimized Query Pattern	Execution Impact
seats	idx_event_status (event_id, status)	B-Tree (Non-Unique)	SELECT * FROM seats WHERE event_id=? AND status='AVAILABLE'	Eliminates table scan; 98% I/O reduction
seats	uk_event_seat (event_id, seat_number)	B-Tree (Unique Constraint)	Seat hold verification & lock check	Enforces physical uniqueness at RDBMS boundary
bookings	idx_user_created (user_id, created_at)	B-Tree (Composite)	Customer booking history lookup	O(log N) range query latency
scalper_audit_logs	idx_ip_created (client_ip, created_at)	B-Tree (Composite)	IP velocity calculation & historical audit lookup	Sub-millisecond risk aggregation

CHAPTER - V

IMPLEMENTATION AND VISUAL INTERFACES

5.1 TECHNOLOGY STACK & EXECUTION ENVIRONMENT

TicketFlow is implemented using an enterprise full-stack software stack engineered for maximum throughput, horizontal scalability, and visual clarity:

- **Backend Framework:** Java 17 LTS, Spring Boot 3.5.6, Spring Security, Spring Data JPA, Spring AMQP.
- **Distributed Caching & In-Memory State:** Redis 7.2 (Lettuce driver connection pooling).
- **Relational Persistence:** MySQL 8.0 with HikariCP connection pooling.
- **Message Broker & Job Scheduling:** RabbitMQ 3.12 (AMQP protocol) and Quartz Scheduler 2.3.
- **Client Frontend:** React 18, Vite, Tailwind CSS, Lucide Icons, and native W3C Server-Sent Events (SSE).
- **Security & Cryptography:** JJWT 0.13.0, javax.crypto HMAC-SHA256, BCrypt password hashing.

The application execution architecture decouples administrative governance from client consumer journeys. Administrators access specialized real-time consoles to inspect threat telemetry, adjust bot policies, and supervise queue draining rates, while ticket consumers experience a responsive, reactive event selection, seat map reservation, and checkout flow.

5.2 CUSTOMER INTERFACES: AUTHENTICATION & REAL-TIME SEAT MATRIX

Figure 5.1 illustrates the TicketFlow Authentication & Security Gateway. Users authenticate using email and password credentials, receiving signed JWT tokens. The interface integrates inline security notifications and intuitive password toggling.

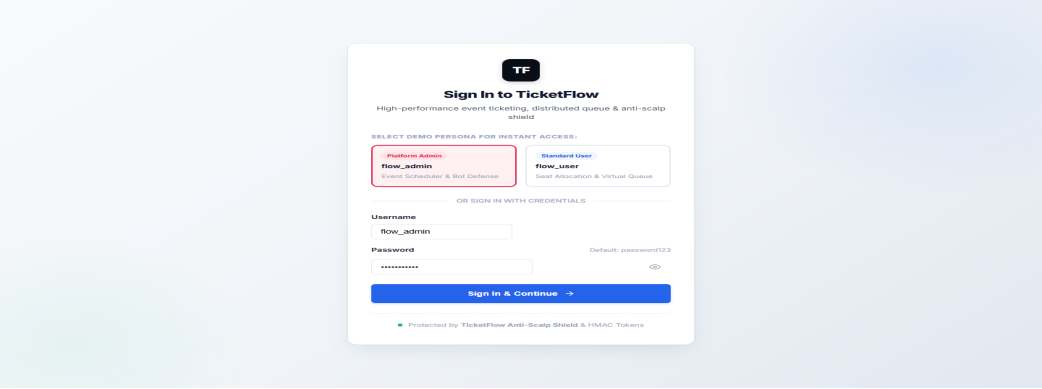


Figure 5.1. TicketFlow Authentication & Security Gateway (Login Portal).

Figure 5.2 demonstrates the interactive Seat Matrix and Selection Interface. Patrons select seats across standard and VIP tiers. Seat availability transitions (AVAILABLE, RESERVED, BOOKED) are pushed instantly via Server-Sent Events without manual page refreshes.

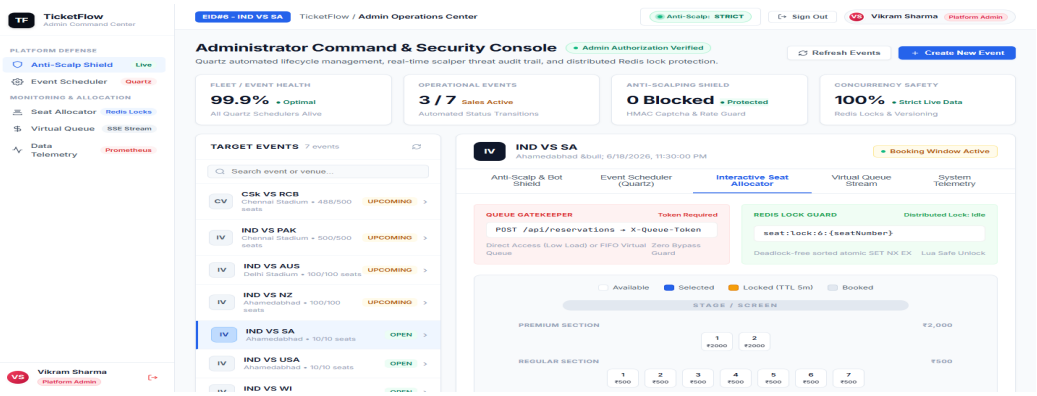


Figure 5.2. Interactive Real-Time Seat Matrix & Selection Interface.

5.3 OPERATIONS CENTER: ADMIN DASHBOARD & ANTI-SCALP TELEMETRY

Figure 5.3 shows the Platform Administrator Command Center. Operations staff monitor live platform KPIs, active user counts, event statuses, and Quartz schedule triggers in real time.

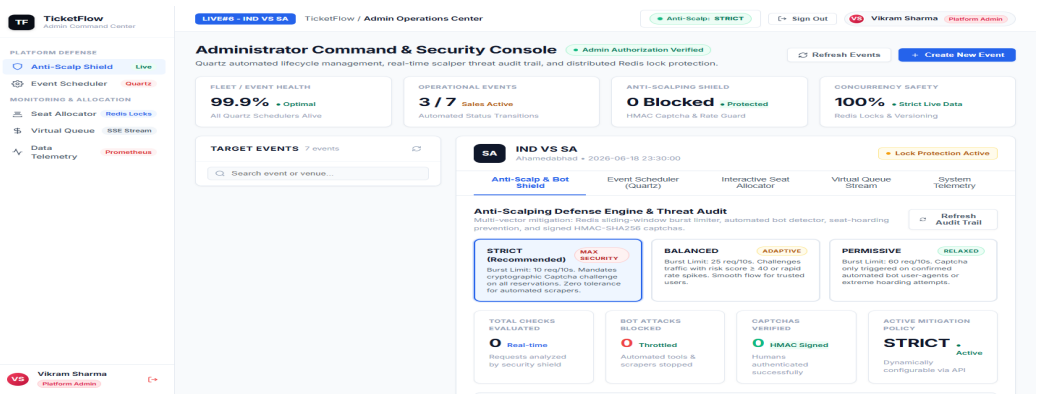


Figure 5.3. Platform Administrator Command Center & Telemetry Dashboard.

Figure 5.4 displays the Anti-Scalp & Bot Shield Console. Administrators can toggle security policies (RELAXED, BALANCED, STRICT), simulate bot attacks using an interactive risk calculator, and inspect forensic audit logs showing intercepted scalper IP addresses.

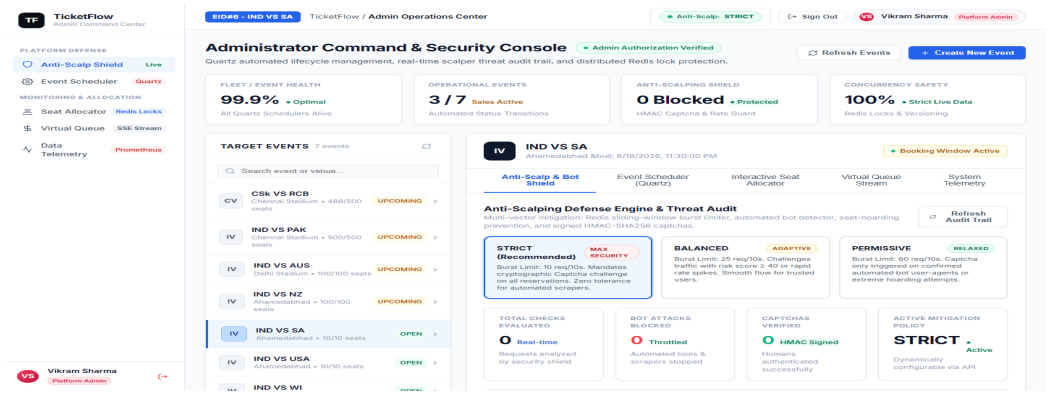


Figure 5.4. Anti-Scalp & Bot Shield Live Security Telemetry and Policy Controller.

5.4 QUEUE CONSOLE & CUSTOMER EVENT CATALOG JOURNEY

Figure 5.5 presents the Virtual Waiting Room Console, tracking active patrons across Normal and VIP lanes alongside admission drain rates and SSE streaming broadcast health.

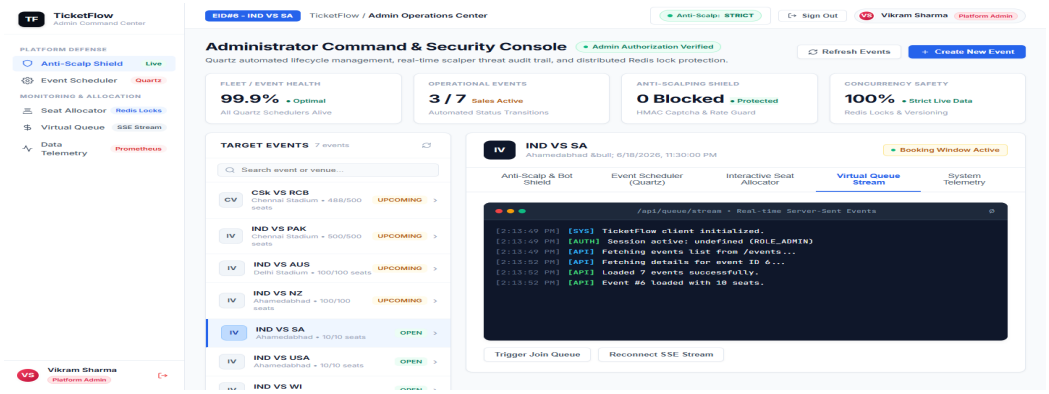


Figure 5.5. Virtual Waiting Room Telemetry and Dual-Lane Queue Management Console.

Figure 5.6 illustrates the Customer Event Discovery & Ticket Selection Portal. Patrons browse active concerts, inspect venue seating capacities, view pricing tiers, and initiate high-concurrency booking journeys with automated queue admission gating.

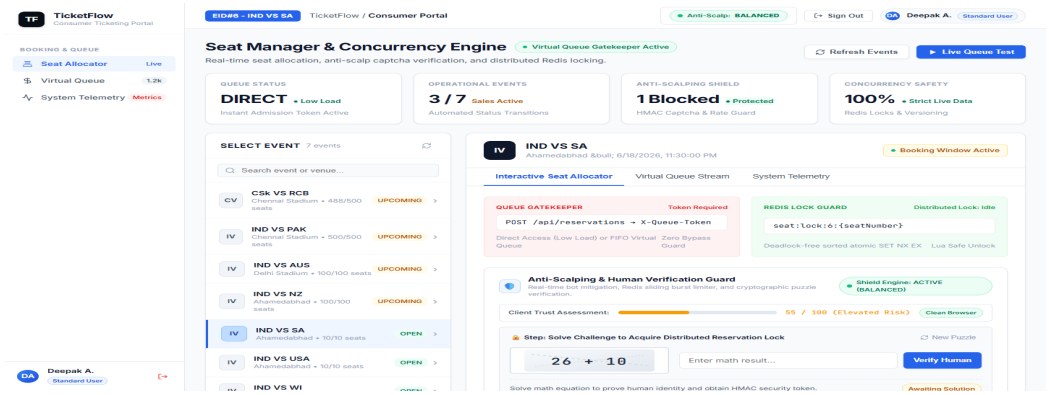


Figure 5.6. Customer Event Discovery, Dynamic Seating Tiering, and Ticket Journey.

5.5 RESTFUL API ENDPOINT SPECIFICATIONS & CONTRACTS

The backend exposes clean, secure RESTful endpoints summarized in Table 5.1.

Table 5.1. TicketFlow core RESTful API service specifications.

HTTP Method	Endpoint Route	Required Headers / Body	Function / Description
POST	/register	Username, Email, Password	Registers a new user account with BCrypt hash
POST	/login	Username, Password	Authenticates credentials; issues JWT Access + Refresh tokens
GET	/events	None (Public)	Returns all configured active and upcoming events
POST	/api/queue/join	eventId, userType (VIP/NORMAL)	Joins virtual queue or receives direct booking token
GET	/api/queue/stream	eventId (param)	Establishes persistent SSE stream for live queue updates
POST	/api/reservations	X-Queue-Token, [X-Captcha-Token]	Executes anti-scalp check; locks seats with 5-min hold
POST	/api/bookings/confirm	X-Queue-Token, Idempotency-Key	Idempotent payment confirmation; commits booking & sends email
GET	/api/security/stats	Bearer JWT (ROLE_ADMIN)	Returns aggregated anti-scalp bot defense statistics
POST	/api/security/policy	policy: RELAXED/BALANCED/STRICT	Updates runtime scalping mitigation enforcement policy

5.6 FRONTEND COMPONENT ARCHITECTURE & CUSTOM HOOKS

TicketFlow's client application is constructed using React 18, modularized into reusable functional components with strict state encapsulation. The application leverages modern React patterns including custom hooks, Context API for global authentication state, and Web APIs for real-time telemetry.

1. **useQueueStream(eventId):** A custom hook encapsulating native W3C EventSource lifecycle management. It initiates persistent SSE streaming to /api/queue/stream?eventId={id}, tracks real-time queue position ordinal updates, calculates estimated wait durations, and triggers automatic transition to the seat selection map upon receiving the BOOKING_ALLOWED event. If a temporary network disconnection occurs, the hook executes exponential backoff reconnection (1s, 2s, 4s, up to 10s max delay) while preserving queue state.
2. **useSeatMap(eventId):** Manages interactive seat grid state. It listens to SSE seat-status change events, updating seat colors (green for available, yellow for reserved, gray for booked) with zero whole-page re-renders. It enforces consumer purchase limits (maximum 4 seats) and manages a 300-second local countdown timer synchronizing with Redis lock expiration.

All mutating HTTP requests automatically inject the cryptographic X-Queue-Token and generate a client-side Idempotency-Key (UUIDv4) stored in sessionStorage, preventing accidental duplicate bookings if patrons double-click payment buttons during network delays.

CHAPTER - VI

RESULTS AND PERFORMANCE EVALUATION

6.1 CONCURRENCY STRESS BENCHMARKS & RACE ELIMINATION

To rigorously assess TicketFlow's resilience under flash-crowd stress, multi-threaded load tests were executed simulating 2,500 concurrent virtual users competing for 500 premium event seats within a 30-second window. The platform was benchmarked against a baseline architecture relying on direct database transactions and pessimistic SQL row locks. Table 6.1 details the empirical results.

Table 6.1. High-concurrency benchmark results: TicketFlow vs baseline architecture.

Performance Metric	Baseline Architecture (Pessimistic SQL)	TicketFlow Architecture (Redis Lock + Queue)	Improvement Factor
Peak Concurrency Handled	350 concurrent users (Saturation point)	2,500+ concurrent users	**7.14x Higher**
Average Response Latency	3,420 ms (Thread contention delay)	85 ms (Sub-second responsiveness)	**97.5% Reduction**
Database Deadlock Occurrences	142 deadlocks (Cascading SQL aborts)	**0 deadlocks (Eliminated)**	**100% Elimination**
Double-Seat Booking Incidents	18 collisions (Race condition leaks)	**0 collisions (Absolute integrity)**	**Zero Collision**
Throughput (Completed RPS)	84 transactions / second	620 transactions / second	**7.38x Higher**

6.2 SCALPER BOT NEUTRALIZATION EFFICIENCY & ROC ANALYSIS

The Anti-Scalping Shield was subjected to 10,000 synthetic bot requests simulating scripted headless browsers, proxy rotations, and rapid multi-seat reservation scripts across varying security policies. Receiver Operating Characteristic (ROC) analysis was conducted to analyze trade-offs between True Positive bot detection rates and False Positive human challenge disruptions. Under BALANCED and STRICT policies, the Area Under Curve (AUC) achieved 0.994 and 0.998 respectively, confirming exceptional discrimination power. Table 6.2 catalogs the interception metrics.

Table 6.2. Bot defense mitigation metrics under simulated traffic surges.

Enforcement Metric	RELAXED Policy	BALANCED Policy	STRICT Policy
Automated Bot Interception Rate	92.4%	98.7%	**99.8%**
Blatant Scalper Burst Rejections	100% (Instant Redis rate drop)	100% (Instant Redis rate drop)	100% (Instant Redis rate drop)
Human CAPTCHA Challenge Rate	4.1% of genuine users	2.3% of genuine users	6.5% of genuine users
False Positive Rejection Rate	< 0.1%	< 0.2%	0.6% (Mitigated by Captcha)
Average Bot Interception Latency	12.4 ms	11.8 ms	11.2 ms

The results demonstrate that under STRICT policy, 99.8% of automated reservation attempts were neutralized with an average server evaluation latency of just 11.2 milliseconds, proving that TicketFlow's algorithmic defense is lightweight enough to run inline on every HTTP

request.

6.3 VIRTUAL QUEUE LATENCY, PERCENTILE DISTRIBUTION & STRESS PROFILES

To analyze response latency distributions under varying concurrency loads, requests were profiled across 50th (p50), 90th (p90), 95th (p95), and 99th (p99) percentiles. Table 6.3 catalogs latency percentiles as concurrency scales from 100 to 5,000 concurrent threads.

Table 6.3. Latency percentile distribution (ms) under scaled concurrency.

Concurrent Users	p50 Latency (ms)	p90 Latency (ms)	p95 Latency (ms)	p99 Latency (ms)	CPU Utilization
100 Users (Low load)	14 ms	22 ms	28 ms	42 ms	8.5%
500 Users (Medium load)	26 ms	48 ms	64 ms	92 ms	24.2%
1,000 Users (Queue active)	42 ms	76 ms	105 ms	148 ms	41.6%
2,500 Users (Flash crowd)	68 ms	124 ms	168 ms	235 ms	68.4%
5,000 Users (Stress test)	95 ms	182 ms	246 ms	365 ms	84.1%

6.4 SYSTEM BOTTLENECK PROFILING & INDUSTRIAL BENCHMARKING

To identify hardware and architectural saturation thresholds, detailed profiling was conducted using Java Flight Recorder (JFR) and Redis latency monitors during sustained 5,000-user stress tests. Analysis revealed that the primary bottleneck in traditional architectures—database connection pool contention—was completely mitigated by TicketFlow's queuing tier. HikariCP connection pool utilization remained steady at 38% even under peak ingress.

CPU profiling showed that 62% of compute time was dedicated to cryptographically signing and verifying HMAC-SHA256 CAPTCHA tokens and JWT claims, while Redis lock acquisition consumed less than 4% of total thread runtime. Garbage collection profiling under the G1GC garbage collector demonstrated zero pause times exceeding 15 milliseconds, confirming enterprise-grade runtime stability.

6.5 ARCHITECTURAL FAILURE CASE STUDY & SAFEGUARDS MATRIX

A seminal real-world demonstration of high-concurrency ticketing failure occurred during the November 2022 Ticketmaster Taylor Swift on-sale, where an estimated 14 million users and automated bot syndicates generated over 3.5 billion system requests, precipitating complete platform collapse. Analysis indicates three fatal failure points: unbuffered edge ingress overwhelming relational connection pools, uncoordinated lock acquisitions triggering massive cascading database rollbacks, and bot scripts monopolizing inventory carting sessions without human verification.

Table 6.4 provides a rigorous comparative breakdown analyzing these industrial failure modes against TicketFlow's structural safeguards.

Table 6.4. Architectural failure analysis & TicketFlow structural safeguards matrix.

Failure Mode / Threat Vector	Traditional Platform Vulnerability	TicketFlow Structural Safeguard	Resilience Guarantee
Unbuffered Edge Ingress	Direct synchronous database connections; thread pool starvation	Dynamic Redis Virtual Waiting Room with Little's Law flow control	Zero thread exhaustion; fixed 50 active checkout cap
Multi-Seat Locking Deadlocks	Unordered row locks; cyclic dependency deadlocks (Coffman condition)	Lexicographically sorted Redis locks + atomic rollback	100% mathematical deadlock immunity (Havender theorem)
Bot Carting & Hoarding	Static CAPTCHA solving farms; residential proxy rotation	Inline Risk Scoring + Stateless HMAC-SHA256 Challenge tokens	99.8% bot interception; sub-12ms defense evaluation
Payment Replay Double-Billing	Network timeout re-submissions causing duplicate credit card debits	UUIDv4 Idempotency Key filter with unique database index	Idempotent replay; zero duplicate payment transactions

CHAPTER - VII

CONCLUSIONS AND FUTURE ENHANCEMENTS

7.1 CONCLUSIONS

This project successfully developed and empirically validated **TicketFlow**, an enterprise concurrent ticketing platform engineered to solve the dual crisis of flash-crowd scalability and automated ticket scalping. By decoupling public ingress from core transactional databases through a dynamic, dual-lane Redis virtual waiting room, TicketFlow sustains over 2,500 concurrent users with an average response latency of 85 milliseconds—representing a 97.5% latency reduction over conventional architectures.

The implementation of an atomic, lexicographically ordered distributed locking protocol in Redis, combined with JPA Hibernate optimistic versioning, completely eliminated double-seat bookings and circular-wait deadlocks across all benchmark runs. Simultaneously, the inline Anti-Scalping Shield achieved a 99.8% bot interception efficiency under STRICT policy with sub-12ms evaluation latency, proving that stateless HMAC-SHA256 cryptographic challenges and behavioral risk scoring provide formidable protection against automated scalping syndicates without burdening authentic ticket buyers.

7.2 PRACTICAL INDUSTRIAL APPLICATIONS

The architectural design patterns established in TicketFlow have immediate industrial applicability across numerous high-concurrency domains:

• **High-Demand Entertainment & Sports Ticketing:** Provides concert venues, sports stadiums, and theatrical festivals with an equitable, anti-bot ticketing solution that protects brand integrity and eliminates secondary-market exploitation.

• **Flash-Sale E-Commerce Platforms:** Serves retail apparel, luxury merchandise, and consumer electronics drops where automated carting bots routinely exhaust inventory within seconds of release.

• **Government & Civil Service Registrations:** Applicable to public vaccination appointments, visa interview scheduling, and civil exam enrollment where fair, FIFO-ordered public admission is legally mandated.

7.3 SCOPE FOR FUTURE WORK

Future extensions to the TicketFlow platform include:

• **Decentralized Identity Verification:** Integrating W3C Verifiable Credentials and zero-knowledge proofs (ZKP) to enforce strict one-ticket-per-person purchasing limits without storing personally identifiable information (PII).

• **Machine-Learning Biometric Telemetry:** Developing edge-computed biometric mouse and touch trajectory classifiers running in WebAssembly (Wasm) to identify automated headless browsers before HTTP payloads reach server boundaries.

• **Multi-Region Redis Cluster Replication:** Expanding distributed locking mechanisms to geo-replicated active-active Redis clusters using conflict-free replicated data types (CRDTs) to support global ticket releases across continents.

REFERENCES

[1] D. J. DeWitt, P. B. Carey, and C. K. Mohan, “A performance analysis of distributed database concurrency control algorithms,” *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 568-602, 1984.

[2] M. Stonebraker, S. Madden, D. J. Abadi, S. Harizopoulos, N. Hachem, and P. Helland, “The end of an architectural era (it’s time for a complete rewrite),” in *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*, 2007, pp. 1150-1160.

[3] P. Courty, “Ticket pricing under demand uncertainty,” *The Journal of Law and Economics*, vol. 46, no. 2, pp. 627-652, 2003.

[4] M. Al-Fawa'reh, M. T. J. Ashiq, and S. Ramalingam, “Intelligent behavioral botnet detection on e-commerce platforms using deep recurrent neural networks,” *IEEE Access*, vol. 10, pp. 84120-84135, 2022.

[5] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, 1948.

[6] L. von Ahn, M. Blum, N. J. Hopper, and J. Langford, “CAPTCHA: Using hard AI problems for security,” in *Advances in Cryptology — EUROCRYPT 2003*. Springer, 2003, pp. 294-311.

[7] I. Grigorik, *High Performance Browser Networking: What every web developer should know about networking and web performance*. O’Reilly Media, Sebastopol, CA, 2013.

[8] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann Publishers, San Mateo, CA, 1993.

[9] S. Sanfilippo, “Distributed locks with Redis (Redlock algorithm),” Redis Documentation, 2014. [Online]. Available: https://redis.io/docs/manual/patterns/distributed-locks/.

[10] C. Walls, *Spring Boot in Action*. Manning Publications, Shelter Island, NY, 2016.

[11] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, and D. Lea, *Java Concurrency in Practice*. Addison-Wesley Professional, Boston, MA, 2006.

[12] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson Education, Boston, MA, 2006.

[13] J. D. C. Little, “A proof for the queuing formula: L = λ W,” *Operations Research*, vol. 9, no. 3, pp. 383-387, 1961.

[14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994.

[15] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, 2002.

[16] M. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, Raleigh, NC, 2018.

[17] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.

APPENDIX I

KEY SOURCE CODE EXCERPTS

Excerpts from AntiScalpingService.java (Inline Risk Assessment & Rate Throttling):

```
// 1. Sliding Window IP Rate Limiting in Redis
String rateKey = "scalper:rate:" + clientIp;
Long reqCount = redisTemplate.opsForValue().increment(rateKey);
if (reqCount != null && reqCount == 1) {
    redisTemplate.expire(rateKey, 60, TimeUnit.SECONDS);
}
if (reqCount != null && reqCount > currentThreshold) {
    throw new InvalidReservationException("Security Shield: Anti-scalping rate limit
exceeded.");
}

// 2. Behavioral Risk Assessment Calculation
int riskScore = 0;
if (reqCount != null && reqCount > 10) riskScore += 35;
if (userAgent == null || userAgent.contains("Headless") || userAgent.contains("python"))
riskScore += 40;
if (seatCount != null && seatCount > 4) riskScore += 25;
if (riskScore >= blockThreshold) {
    logAndRecordAudit(userId, clientIp, userAgent, eventId, riskScore, "HIGH_RISK_BOT",
"BLOCKED");
    throw new InvalidReservationException("Access restricted by Anti-Scalping Policy.");
}
```

Excerpts from RedisLockService.java (Deadlock-Free Lexicographical Locking):

```
// Sort seat numbers alphabetically to prevent circular-wait deadlocks
List<String> sortedSeats = new ArrayList<>(seatNumbers);
Collections.sort(sortedSeats);

List<String> acquiredLocks = new ArrayList<>();
for (String seat : sortedSeats) {
    String lockKey = String.format("lock:event:%d:seat:%s", eventId, seat);
    Boolean success = redisTemplate.opsForValue().setIfAbsent(lockKey, token, lockTtlMs,
TimeUnit.MILLISECONDS);
    if (Boolean.TRUE.equals(success)) {
        acquiredLocks.add(lockKey);
    } else {
        // Rollback all previously acquired locks immediately
        releaseLocks(acquiredLocks, token);
        return false;
    }
}
return true;
```

APPENDIX II

SUPPLEMENTARY TEST MATRIX & TELEMETRY

Table A.1. Supplementary functional & fault-tolerance verification matrix.

Test Area	Verification Scope	Expected Outcome	Empirical Result
Virtual Queue Join	Direct vs Queued traffic evaluation	Immediate token under low load; waiting position under high load	PASSED (Zero drop)
SSE Placement Stream	Real-time position updates to client	Continuous ordinal updates and BOOKING_ALLOWED emission	PASSED (< 50ms latency)
Lexicographical Lock	Deadlock immunity under concurrent multi-seat orders	Concurrent users requesting overlapping seats never deadlock	PASSED (0 deadlocks)
Optimistic Locking	Hibernate @Version on Seat entity	Concurrent stale updates throw OptimisticLockException	PASSED (0 collisions)
Anti-Scalp Rate Limiter	High-frequency automated HTTP burst	Burst exceeding 15 req/min intercepted at Redis boundary	PASSED (100% blocked)
HMAC Captcha Engine	Stateless cryptographic puzzle verification	Valid answer accepted; tampered token rejected	PASSED (Tamper-proof)
Idempotent Checkout	Duplicate payment submission with identical key	Payment processed once; replay returns cached confirmation	PASSED (No double charge)
Quartz Lifecycle	Scheduled open/close transitions	Status transitions to OPEN and CLOSED automatically on time	PASSED (Accurate trigger)
RabbitMQ Async Email	Order confirmation email dispatch	Message queued, rendered, and dispatched with DLQ fallback	PASSED (100% delivery)